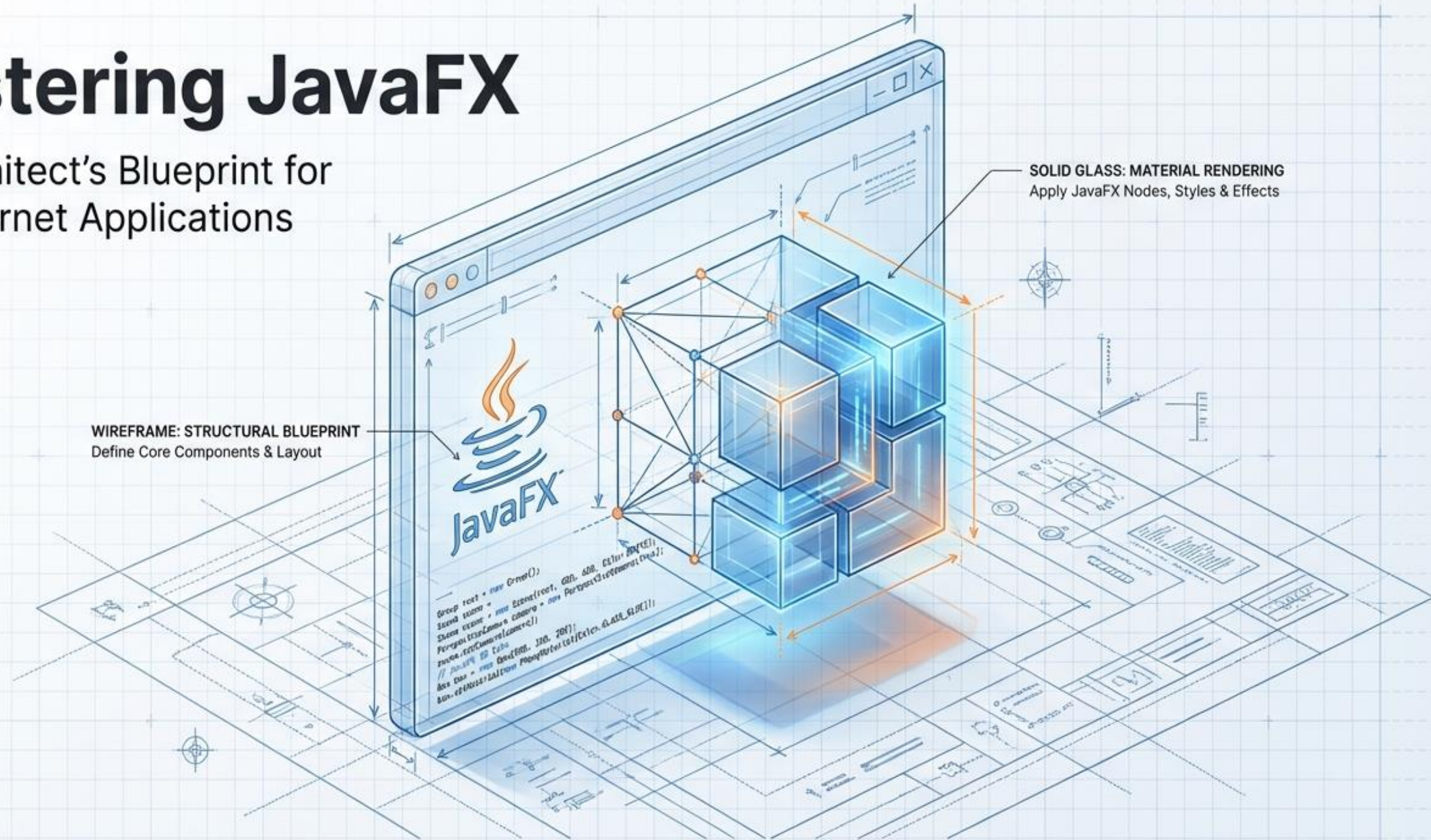


Mastering JavaFX

The Architect's Blueprint for Rich Internet Applications

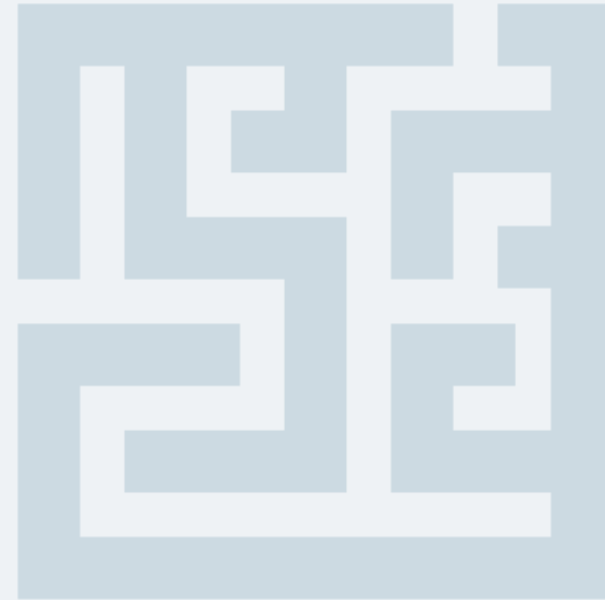




Advanced Programming Applications

Lecture 05- JavaFX – Part II

Dr. Mohamed Mostafa Eltaweel



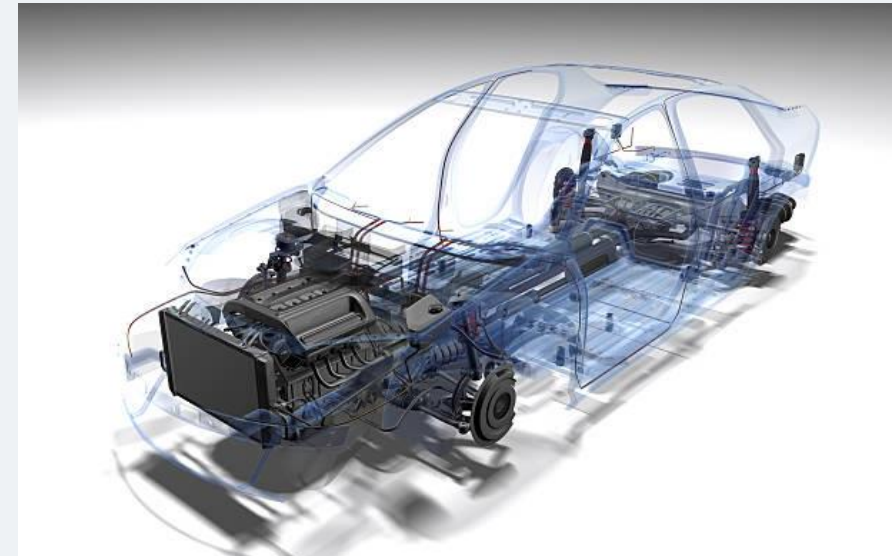
Lecture Outline

1. Inner Classes
2. Anonymous Inner Classes
3. Lambda Expression
4. JavaFX - Event Handling
5. JavaFX - Animations
6. JavaFX and CSS
7. Practice

Inner Classes

- **Inner class:** A class is a member of another class.
- **Advantages:** In some applications, you can use an inner class to make programs simple.
 - An inner class can reference the data and methods defined in the outer class in which it nests, so you do not need to pass the reference of the outer class to the constructor of the inner class

```
public class Car {// The Outer Class  
  
    public class Engine {// THIS IS THE INNER CLASS  
  
        } // END OF THE INNER CLASS  
}
```

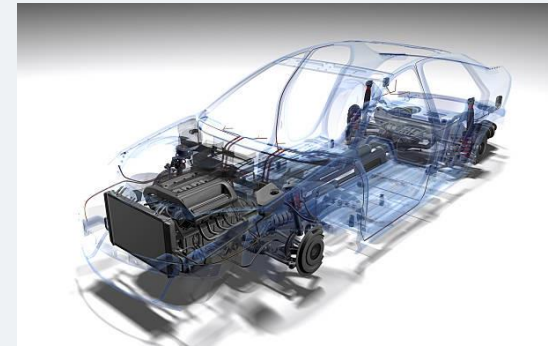


```
public class Car {// The Outer Class
    private String model;
    private boolean isRunning;

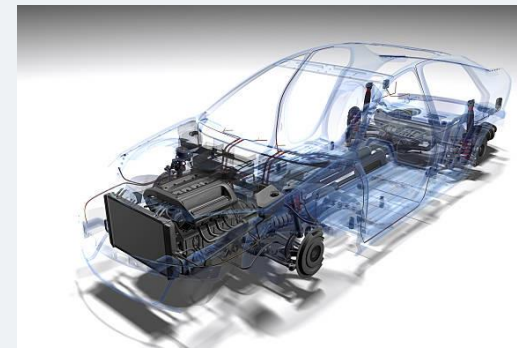
    public Car(String model) {
        this.model = model;
        this.isRunning = false;
    }
}
```

```
public class Engine {// THIS IS THE INNER CLASS
```

```
    public void start() {
        // The Inner Class can access the Outer Class's private variables!
        if (!isRunning) {
            isRunning = true;
            System.out.println("The " + model + "'s engine fires up. Vroom!");
        }
    }
} // END OF THE INNER CLASS
}
```




```
public class Main {  
    public static void main(String[] args) {  
  
        // Step 1: You MUST create an instance of the Outer Class first  
        Car myMustang = new Car("Ford Mustang");  
  
        // Step 2: Use the Outer Class instance to create the Inner Class  
        // Notice the syntax: OuterClass.InnerClass variable = outerInstance.new InnerClass();  
  
        Car.Engine mustangEngine = myMustang.new Engine();  
  
        // Step 3: Run the inner class method  
        mustangEngine.start(); // Output: The Ford Mustang's engine fires up. Vroom!  
        mustangEngine.start(); // Output: The Ford Mustang's engine is already running.  
    }  
}
```



Inner Classes

- An inner class supports the work of its containing outer class and is compiled into a class named
OuterClassName\$InnerClassName.class
- An inner class can be declared public, protected, or private subject to the same visibility rules applied to a member of the class.
- An inner class can be declared static.
 - A static inner class can be accessed using the outer class name.
 - A static inner class cannot access nonstatic members of the outer class.

Anonymous Inner Classes

- An *anonymous inner class* is an inner class without a name
- An anonymous inner class **must always extend** a superclass or implement an interface, **but it cannot have an explicit extends or implements clause.**
- An anonymous inner class must implement **all the abstract methods** in the superclass or in the interface.
- An anonymous inner class uses the no-arg constructor from its superclass to create an instance.
- An anonymous inner class is compiled into a class named OuterClassName\$n.class. For example, if the outer class Test has two anonymous inner classes, these two classes are compiled into Test\$1.class and Test\$2.class.

Anonymous Inner Classes

**The Traditional
Way:
A Named Class**

```
interface MathOperation {
    int calculate(int a, int b);
}

class AdditionOperation implements MathOperation {
    @Override
    public int calculate(int a, int b) {
        int sum = a + b;
        System.out.println("The sum is: " + sum);
        return sum;
    }
}

public class Main {
    public static void main(String[] args) {
        MathOperation addition = new AdditionOperation();
        addition.calculate(5, 10);
    }
}
```

Anonymous Inner Classes

```
interface MathOperation {  
    int calculate(int a, int b);  
}
```

```
class AdditionOperation{
```

**The Modern Way:
Anonymous Inner
Class**

```
public class Main {  
    public static void main(String[] args) {  
  
        MathOperation addition = new MathOperation() {  
  
            @Override  
            public int calculate(int a, int b) {  
                int sum = a + b;  
                System.out.println("The sum is: " + sum);  
                return sum; }  
        };  
        addition.calculate(5, 10);  
    }  
}
```

Lambda Expressions

- *Lambda expression* is a new feature in Java 8.
- Lambda expressions can be viewed as an
- Lambda expression facilitates **functional programming**.
- A Lambda expression is a short block of code that takes in parameters and returns a value.
- It is essentially an **anonymous function**—a method that doesn't need a name, an access modifier (like public or private), or a return type declaration.

parameter -> expression

(parameter1, parameter2) -> expression

Lambda Expressions

- The Secret Sauce: Functional Interfaces
- You can't just put a lambda expression anywhere. They can *only* be used in contexts where a **Functional Interface** is expected.

parameter -> expression

(parameter1, parameter2) -> expression

Check the rest of expressions at
https://www.w3schools.com/java/java_lambda.asp

Lambda Expressions

```
Button btn = new Button("Submit");

btn.setOnAction(new EventHandler<ActionEvent>()
{
    @Override
    public void handle(ActionEvent event) {
        System.out.println("Button was
clicked!");
    }
});
```

"Before"

"After"

```
Button btn = new Button("Submit");

btn.setOnAction(event -> System.out.println("Button was clicked!"));
```

Lambda Expressions

Write a Lambda expression for adding two integers

```
(a, b) -> {  
    int sum = a + b;  
    System.out.println("The sum is: " + sum);  
  
    return sum;  
};
```


Lambda Expressions

Programming Exercise:

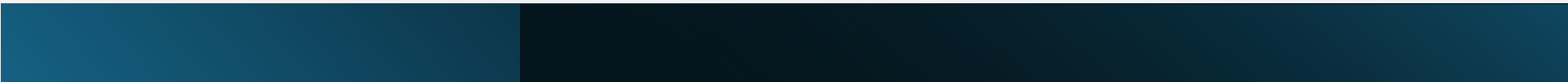
Define the Interface: Create an interface named Calculator. Use the appropriate annotation to mark it as a functional interface.

Declare the Method: Inside the interface, define a single abstract method named sum that takes two int parameters and returns an int.

Setup the Main Class: Create a Main class containing a standard public static void main(String[] args) method.

Implement with a Lambda: Inside the main method, declare a variable of type Calculator. Assign it a Lambda expression that correctly implements the sum method by adding the two integer parameters together.

Execute and Print: Call the sum method using your lambda implementation, passing in the values 15 and 25. Store the result and print it to the console.



Lambda Expressions

Write a Lambda expression for adding two integers

```
interface Calculator {  
    int sum(int a, int b);  
}  
  
public class CalcMain {  
    public static void main(String[] args) {  
  
        // Use a Lambda expression to implement the interface  
        // Java knows 'a' and 'b' are integers because of the interface definition  
        Calculator addition = (a, b) -> a + b;  
  
        // 3. Call the method and store/print the result  
        int result = addition.sum(15, 25);  
  
        System.out.println("The result of the sum is: " + result);  
    }  
}
```

The Secret Sauce: Functional Interfaces

- You can't just put a lambda expression anywhere. They can *only* be used in contexts where a **Functional Interface** is expected.
- A Functional Interface is simply any interface in Java that contains **exactly one abstract method**.
 - ✓ **EventHandler** <T> is a functional interface because it only has handle().
 - ✓ **Runnable** is a functional interface because it only has run().
 - ✓ **ActionListener** is a functional interface because it only has actionPerformed().

JavaFX - Event Handling

JavaFX - Event Handling

- An **event** occurs whenever a **user interacts with application components (nodes)**.
- **Examples of Events:**
 - Clicking a button
 - Moving the mouse
 - Typing using the keyboard
 - Selecting an item from a list
 - Scrolling a page

Main Types of Events

1. Foreground Events

- Require **direct user interaction**.
- Generated when users interact with GUI elements.
- **Examples:**
 - Button click
 - Mouse movement
 - Keyboard input
 - Selecting items from lists
 - Scrolling

2. Background Events

- **Do not require user interaction.**
- Triggered by the system or application processes.
- **Examples:**
 - Operating system interruptions
 - Hardware or software failures
 - Timer expiration
 - Completion of an operation

The class named **Event** of the package **javafx.event** is the base class for an event.

Event Handling in JavaFX

- Event handling is the **mechanism that controls events and determines what action should occur when an event happens.**
- **Event Handler:**
 - The **code that executes when an event occurs.**
 - It defines the response of the application to user actions.

Key Components of a JavaFX Event

1. Target

- The **node** where the event occurs.
- Examples: **Window, Scene, or Node** (such as **Button, TextField, etc.**)

2. Source

- The **object** that generates the event.
- Example: **Mouse or Keyboard** generating the event.

3. Type

- The **specific category of the event**.
- Example for mouse events:
 - **Mouse Pressed**
 - **Mouse Released**


```
public class TargetExample extends Application {
```

Example: Button as the Target

```
@Override
```

```
public void start(Stage stage) {
```

```
    Button btn = new Button("Click Me");
```

```
    btn.setOnMouseClicked(e -> {
```

```
        System.out.println("Target: " + e.getTarget());  
        System.out.println("Source: " + e.getSource());  
        System.out.println("Event Type: " + e.getEventType());
```

```
    });
```

```
    Scene scene = new Scene(btn, 300, 200);
```

```
    stage.setScene(scene);
```

```
    stage.setTitle("Target Example");
```

```
    stage.show();
```

```
}
```

```
public static void main(String[] args) {
```

```
    launch(); } }
```

Target

Type: Mouse
Clicked

Source

Outputs:

Target: Button

Source: Button

Event Type: MOUSE_CLICKED

Examples of JavaFX EventHandler

User Action	Event Type	EventHandler Properties
Pressing, releasing, or typing a key on keyboard.	KeyEvent	onKeyPressed onKeyReleased onKeyTyped
Moving, Clicking, or dragging the mouse.	MouseEvent	onMouseClicked onMouseMoved onMousePressed onMouseReleased onMouseEntered onMouseExited
Pressing, Dragging, and Releasing of the mouse button.	MouseEvent	onMouseDragged onMouseDragEntered onMouseDragExited onMouseDragged onMouseDragOver onMouseDragReleased
Generating, Changing, Removing or Committing input from an alternate method.	InputMethodEvent	onInputMethodTextChanged
Performing Drag and Drop actions supported by the platform.	DragEvent	onDragDetected onDragDone onDragDropped onDragEntered onDragExited onDragOver
Scrolling an object.	ScrollEvent	onScroll onScrollStarted onScrollFinished
Zooming on an object.	ZoomEvent	onZoom onZoomStarted onZoomFinished

Source:
[TutorialsPoints](#)

Example

```
playButton.setOnMouseClicked((new EventHandler<MouseEvent>() {  
  
    public void handle(MouseEvent event) {  
        System.out.println("Hello World");  
        pathTransition.play();  
    }  
  
})) ;
```

JavaFX - Animations

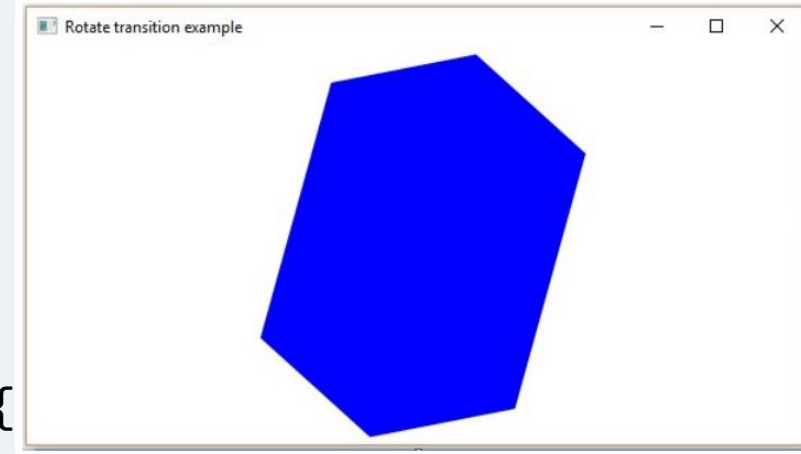
- In JavaFX, a node can be animated by changing its property over time. JavaFX provides a package named **javafx.animation**

Applied animations (transitions) such as –

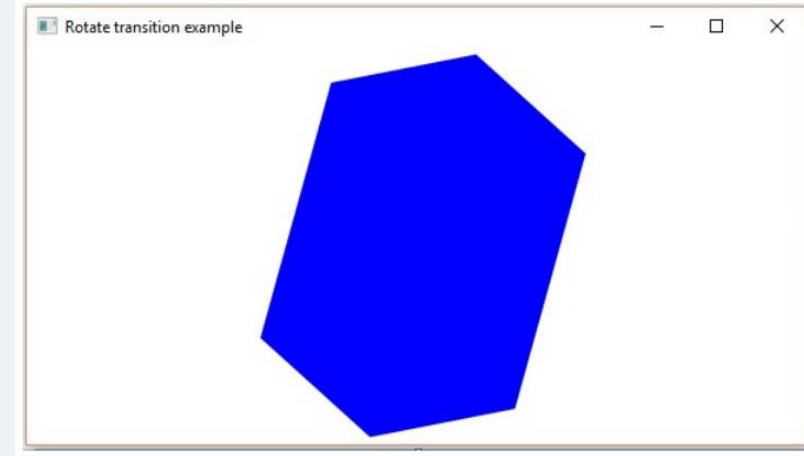
- **Fade Transition**
- **Fill Transition**
- **Rotate Transition**
- **Scale Transition**
- **Stroke Transition**
- **Translate Transition**
- **Path Transition**
- **Sequential Transition**
- **Pause Transition**
- **Parallel Transition**

Rotate Transition in JavaFX

```
public class RotateTransitionExample extends Application {  
    @Override  
    public void start(Stage stage) {  
        //Creating a hexagon  
        Polygon hexagon = new Polygon();  
  
        //Adding coordinates to the hexagon  
        hexagon.getPoints().addAll(new Double[]{  
            200.0, 50.0,  
            400.0, 50.0,  
            450.0, 150.0,  
            400.0, 250.0,  
            200.0, 250.0,  
            150.0, 150.0,  
        });  
        //Setting the fill color for the hexagon  
        hexagon.setFill(Color.BLUE);  
    }  
}
```



Rotate Transition in JavaFX



```
//Creating a rotate transition
RotateTransition rotateTransition = new RotateTransition();

//Setting the duration for the transition
rotateTransition.setDuration(Duration.millis(1000));

//Setting the node for the transition
rotateTransition.setNode(hexagon);

//Setting the angle of the rotation
rotateTransition.setByAngle(360);

//Setting the cycle count for the transition
rotateTransition.setCycleCount(50);

//Setting auto reverse value to false
rotateTransition.setAutoReverse(false);
```

Source:
[Tutorialspoints](https://www.tutorialspoint.com/javafx/rotate_transition_in_java_fx.htm)

Rotate Transition in JavaFX

```
//Playing the animation
rotateTransition.play();

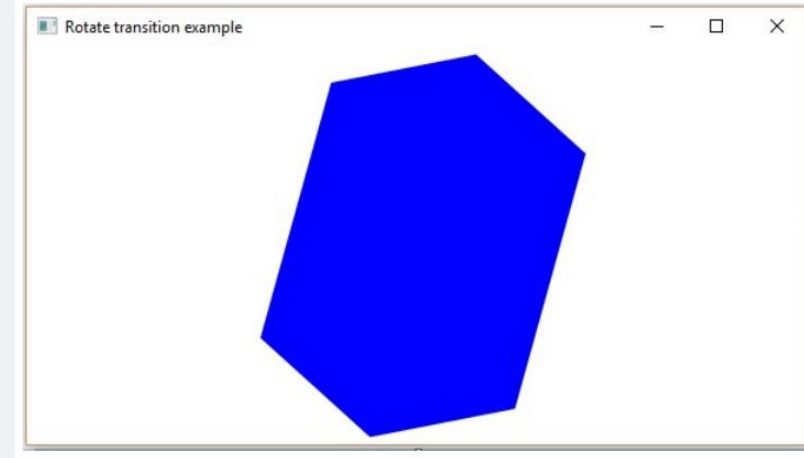
//Creating a Group object
Group root = new Group(hexagon);

//Creating a scene object
Scene scene = new Scene(root, 600, 300);

//Setting title to the Stage
stage.setTitle("Rotate transition example ");

//Adding scene to the stage
stage.setScene(scene);

//Displaying the contents of the stage
stage.show();
}
```

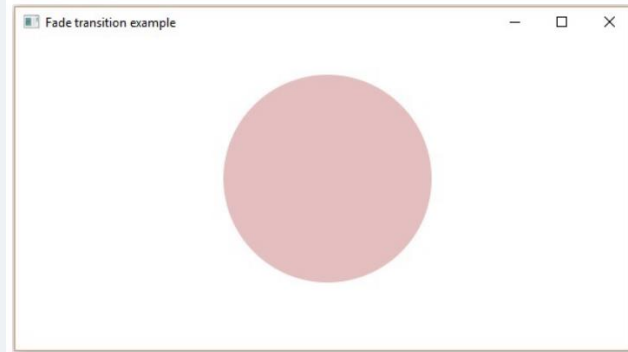


JavaFX - Fade Transition

```
import javafx.animation.FadeTransition;
import javafx.application.Application;
import javafx.scene.Group;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.stage.Stage;
import javafx.util.Duration;

public class FadeTransitionExample extends Application {
    @Override
    public void start(Stage stage) {
        //Drawing a Circle
        Circle circle = new Circle();

        //Setting the position of the circle
        circle.setCenterX(300.0f);
        circle.setCenterY(135.0f);
```



JavaFX - Fade Transition

```
//Setting the radius of the circle
circle.setRadius(100.0f);

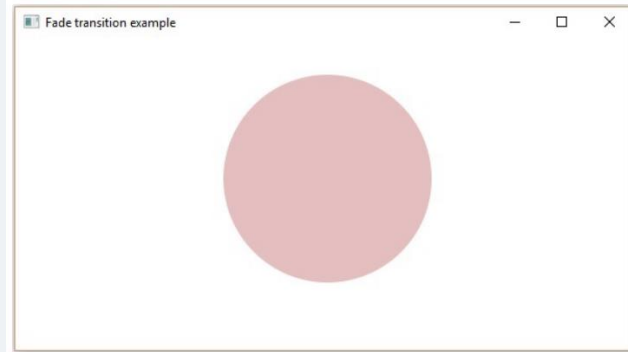
//Setting the color of the circle
circle.setFill(Color.BROWN);

//Setting the stroke width of the circle
circle.setStrokeWidth(20);

//Creating the fade Transition
FadeTransition fadeTransition = new
FadeTransition(Duration.millis(1000));

//Setting the node for Transition
fadeTransition.setNode(circle);

//Setting the property fromValue of the transition (opacity)
fadeTransition.setFromValue(1.0);
```



JavaFX - Fade Transition

```
//Setting the property toValue of the transition (opacity)
fadeTransition.setToValue(0.3);

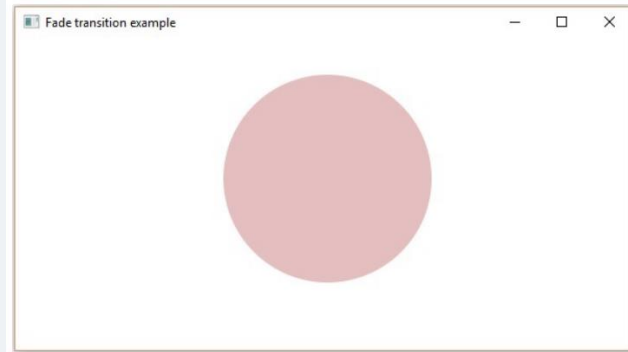
//Setting the cycle count for the transition
fadeTransition.setCycleCount(50);

//Setting auto reverse value to false
fadeTransition.setAutoReverse(false);

//Playing the animation
fadeTransition.play();

//Creating a Group object
Group root = new Group(circle);

//Creating a scene object
Scene scene = new Scene(root, 600, 300);
```

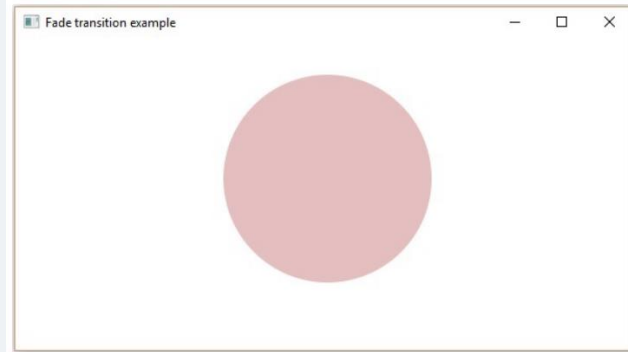


JavaFX - Fade Transition

```
//Setting title to the Stage
stage.setTitle("Fade transition example");

//Adding scene to the stage
stage.setScene(scene);

//Displaying the contents of the stage
stage.show();
}
public static void main(String args[]){
    launch(args);
}
}
```



JavaFX and CSS

- we can add our own style sheet to a scene in JavaFX using the **getStylesheets()** method as shown below

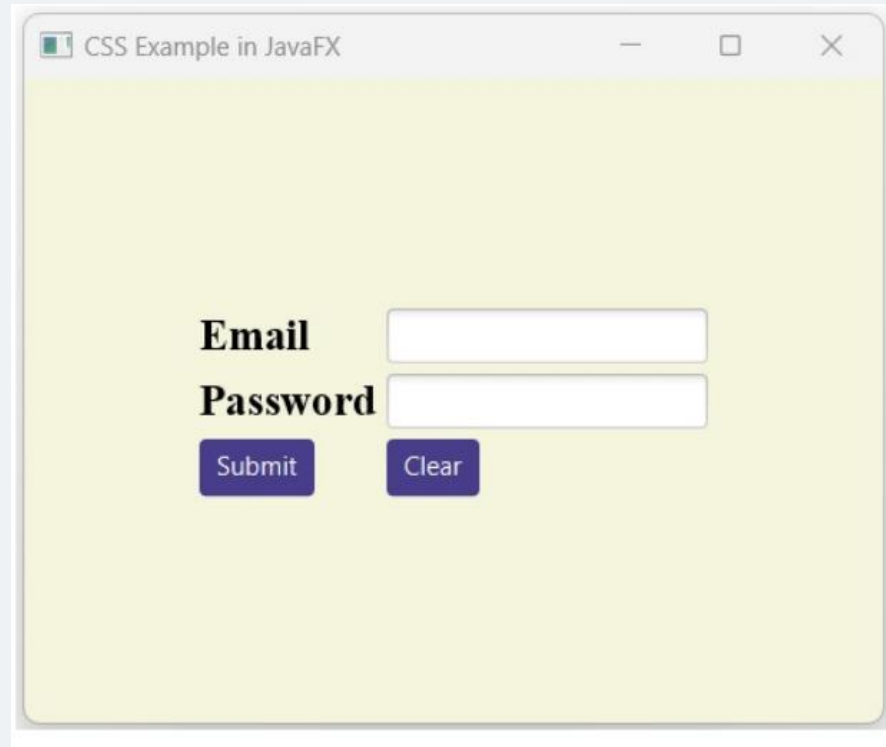
```
Scene scene = new Scene(new Group(), 500, 400);  
scene.getStylesheets().add("path/stylessheet.css");
```

- we can also add in-line styles using the **setStyle()** method.

```
text1.setStyle("-fx-font: normal bold 20px 'serif' ");  
text2.setStyle("-fx-font: normal bold 20px 'serif' ");
```

Assignment

- Use JavaFx to generates a fully functional window as shown below.
- The password and user name are stored in text file



The image shows a JavaFX application window titled "CSS Example in JavaFX". The window has a light yellow background. It contains a login form with the following elements:

- Email**: A text label followed by a white text input field.
- Password**: A text label followed by a white text input field.
- Submit**: A blue button with white text.
- Clear**: A blue button with white text.



Thank you

